

AI 활용 기술 문서 — INCANT

제출물 ④ · NHN 2026 게임 AI 해커톤 INCANT: "말이 곧 마법" — 플레이어가 자유롭게 타이핑한 문장을 LLM이 실시간으로 해석해 주문으로 바꾸는 로그라이크.

이 문서는 대회 제출 요건 ④(AI 활용 기술적 설명 / AI 도구 활용 내역 / 외부 에셋·오픈소스 출처)에 대응한다.

기술 기준일 2026-08-09: 최신 main 의 meaning-v2.31-ultimate-resonance-echo 판정 계약과 코드·회귀 검사를 대조해 작성했다. 폐기된 플레이어 move 안무는 현재 기능과 분리해 설계 변경 이력으로만 서술한다.

목차

- **1부. 게임 속 AI** — 제품에 탑재된 AI의 구조와 프롬프트
 - 1.1 개요: LLM이 게임에서 말하는 일
 - 1.2 판정 파이프라인
 - 1.3 판정 프롬프트 전문
 - 1.4 "LLM을 신뢰하지 않는" 설계
 - 1.5 행동 설계의 경계: 소환 behavior·벽 shape-form/wait 시간축
 - 1.6 기억하는 보스: 도발 대사 + 미러 캐스트
 - 1.7 진화·융합 작명
 - 1.8 인프라: Cloudflare Worker 프록시
- **2부. AI로 만든 게임** — 개발에 사용한 AI 도구와 활용 내역
 - 2.1 사용한 AI 도구
 - 2.2 협업 워크플로
 - 2.3 대표 사례: 프롬프트에서 산출물까지
 - 2.4 배운 점: 에이전트 지휘 노하우
- **3부. 외부 에셋·오픈소스 출처**
 - 3.1 오디오 / 3.2 이미지 / 3.3 폰트 / 3.4 자체 제작 / 3.5 오픈소스

1부. 게임 속 AI

1.1 개요 — LLM이 게임에서 말하는 일

INCANT의 핵심 경험은 **정해진 스킬 목록이 없다는 것**이다. 플레이어는 "얼음 감옥으로 가둬라", "숲의 분노", "lightning storm", "배고프다" 같은 **어떤 문장이든** 입력할 수 있고, 게임은 그 의미를 해석해 원소·형태·위력·효과를 갖춘 실제 주문으로 변환한다.

이 변환의 심장이 ****주문 판정기(judge)****다. 판정기는 자유 텍스트를 받아 게임 엔진이 소비할 수 있는 **구조화된 JSON**을 돌려준다.

플레이어 입력: "얼음 가시로 꿰뚫어라"

|

▼ (LLM 판정)

```
구조화 판정: {
  disposition: "cast",
  spell_plan: {
    name: "빙결의 가시", power: 62, durationMs: 80,
    sequences: [{ behaviors: [{ type: "form", spec: {
      effect: "damage", target: "enemy", element_primary: "ice",
      element_secondary: null, form: "bolt", size: "medium",
      speed: "fast", status: ["freeze"], power: 0, cost: 0
    }}}]}
  }
}
```

|

▼

게임 엔진: 전체 위력 62의 예산을 검증·배분해 얼음 bolt와 빙결 상태이상을 실행

판정에 사용하는 모델은 **Google Gemini 3.5 Flash-Lite**이며, 클라우드플레이어 워커 프록시를 통해 호출한다. 판정 외에도 두 곳에서 LLM을 쓴다 — **보스의 도발 대사 생성(1.6)**과 **주문 진화·정령 융합 시의 작명(1.7)**.

설계 원칙: LLM은 "의미를 이해하는 번역기"로만 쓰고, **게임 밸런스·안전·재현성은 코드가 통제한다**. LLM의 출력은 항상 검증·클램프를 거치며, 실패해도 게임은 멈추지 않는다(1.4).

1.2 판정 파이프라인

판정 한 번은 다음 체인을 거친다. 각 단계는 "LLM에 도달하기 전에 최대한 거르고, LLM이 실패해도 게임은 계속 돌게" 설계 됐다. (구현: `src/spell/geminiJudge.ts`)

```
judge(text)
├─ ① 로컬 사전검사 (precheck) — 명백한 넌센스·차단어는 LLM 호출 없이 즉시 처리
│   └─ mockJudge.precheckText() source: 'local'
├─ ② localStorage 캐시 조회 — 같은 문장 = 같은 판정 (재현성·속도·호출량 절감)
│   └─ 키: incant:judge:v2:meaning-v2.31-ultimate-resonance-echo:
│       <일반/필살>:<공명 컨텍스트>:<문장> source: 'cache'
├─ ③ 프록시 요청 — Cloudflare Worker → Gemini 3.5 Flash-Lite
│   └─ POST { text, castMode, resonance? }
│       시도당 2.5초, timeout일 때만 즉시 1회 재시도(총 원격 예산 최대 5초)
├─ ④ 스키마 재검증 (validateJudgement) — 스키마 밖 값은 거부·클램프
│   └─ 유효하면 캐시에 저장 (장애 폴백은 저장하지 않음)
└─ ⑤ 폴백 (MockJudge) — 타임아웃·네트워크 오류·무효 응답 시
    └─ 게임은 절대 멈추지 않는다 source: 'fallback'
```

핵심 설계점:

- **시도별 2.5초 + timeout 전용 1회 재시도** (`JUDGE_ATTEMPT_TIMEOUT_MS = 2500`, `JUDGE_MAX_ATTEMPTS = 2`): 일시적인 지연이면 새 요청으로 한 번 복구하고, 최대 5초 안에도 응답이 없으면 로컬 폴백으로 전환한다. HTTP 오류·JSON 오류는 같은 요청을 반복하지 않는다.
- **캐시 프리위밍:** 시연 코퍼스의 판정을 사전 계산해 로드 시 localStorage에 주입 → 심사위원 첫 영창이 0ms + 라이브 호출 ↓ (무료 티어 할당량 완화). 버전 불일치·기존 캐시·fizzle은 자동 제외(`seedJudgeCache`).
- **캐시는 유효 결과만 저장:** 장애로 인한 폴백 판정은 캐시하지 않는다. 프록시가 복구되면 다음 시전에서 진짜 판정을 받는다.
- **출처 추적 (lastSource):** 각 판정이 `local / cache / gemini / fallback` 중 어디서 왔는지 기록해, 개발 중 폴백 빈도를 관찰할 수 있게 했다.

- **런 단위 오프라인 복구 (RunRecoveryJudge)**: 한 런에서 연속 2회 원격 폴백이 나면 이후 영창은 즉시 로컬 판정한다. 백그라운드 probe가 성공하면 다음 영창부터 원격 판정을 다시 사용해, 장애 중 매번 5초를 기다리는 일을 막는다.

1.3 판정 프롬프트 전문

프롬프트는 **서버(프록시)에 고정**한다. 클라이언트는 플레이어 입력과 판정 모드인 { text, castMode, resonance? } 만 보내고, 판정 규칙·출력 스키마는 워커가 강제한다 — 프롬프트를 클라이언트에 두면 조작이 가능하기 때문이다(1.8).

프롬프트는 LLM에게 **정해진 순서로 판단**하도록 지시한다. 아래는 현재 배포본 **meaning-v2.31-ultimate-resonance-echo** 전문의 핵심 골자다(전체 전문은 `proxy/worker.js`).

당신은 자유 텍스트 마법 게임의 의미 판정관이다. 반드시 JSON 하나만 출력한다.

다음 순서를 지켜 판단한다.

1. 입력 언어와 표현의 실제 의미를 파악한다. 외래어와 비유도 번역해 이해한다.
2. disposition 결정 – cast(의미 있는 모든 문장, 완결된 동작·서술문도 포함) / fizzle(의미 없는 키보드 메시지) / blocked(욕설·혐오만).
3. cast면 effect·target을 먼저 정하고 element·form을 시각적 은유로 고른다.
 - "배고프다"=heal/self, "나를 지켜줘"=shield/self, "라이트닝 스톰"="lightning storm".
 - 근접에서 베거나 휘두르는 동작(칼·도끼·발톱·참격)은 form=slash – 투사체(bolt)·광선(beam)과 구분.
4. 일반 영창의 전체 power(30~80)와 cost(power의 0.5~0.7배)를 정한다.
 - 표기 언어(한국어·외래어·영어)는 power에 영향 없다 – 번역해 의미·구체성이 같으면 power도 같다.
 - 문장 길이·원소 수·form 수가 아니라 사건·관계·변화가 실제로 연결될 때만 제한적으로 가정한다.
5. effect가 summon이면 소환수 움직임을 behavior(6종 kind)로 설계한다 (→ 1.5).
6. form이 wall이면 벽 모양을 shape(6종 kind)로 설계한다 (→ 1.5).
7. 모든 cast는 대표 spell 대신 ****spell_plan****으로 출력한다. 단일 사건은 form 하나의 원자 plan, 순서·동시·반복·지연·변화가 읽히면 그 관계를 보존한 복합 plan으로 구성한다.
 - 안무 단위: sequences(순차, 최대 10) × behaviors(같은 순간 병렬, 최대 5).
 - behavior type은 form(실제 효과)·wait(사건 사이 간격) 둘뿐이다. player move는 출력하지 않는다.
 - 일반 영창은 최대 2원소만 사용하며, 입력에 없는 사건·effect·원소를 화려함 때문에 추가하지 않는다.
 - plan.power는 전체 예산이고 각 form은 powerWeight로 나눠 쓴다. spec.power·cost는 0으로 둔다.
8. 필살영창은 별도 계약이다. power 100, 4~8 sequences, 6~12 forms, 4~6초로 확장하고 현재 문장이 주도권을 가진 채 이번 게임지에 기여한 원소·형태·주문명 중 1~2개만 echo로 반영한다.

(예: "빛의 창" → beam 하나의 원자 plan · "종이 두 번 울리고 잠시 뒤 다시 울린다" → form → wait → form · "일식의 왈츠" → 빛·어둠 form의 병렬 교차와 필요할 때만 피날레. 진화 이력은 `R2_SYSTEM_GUIDE.md`.)

이 프롬프트가 담은 설계 의도:

- **의미 우선, 단어 매칭 아님** (1단계): "화염구" 든 "fireball" 이든 "불덩이를 던져라" 든 같은 의미로 해석한다. 외래어·비유·다국어 모두 번역해 이해하도록 지시했다.
- **넌센스와 유해 입력만 거부** (2단계): 의미가 조금이라도 있으면 cast 한다. 키보드 메시지("asdf")만 fizzle, 욕설·혐오만 blocked. "우리가 미리 정한 마법 단어"에 갇히지 않게 하는 것이 핵심.
- **언어 중립 위력** (4단계): 한국어로 쓰든 영어로 쓰든, **의미와 구체성이 같으면 power도 같아야 한다**. 특정 언어 사용자가 유·불리하지 않도록 명시했다. (이 규칙은 held-out 검증으로 교차언어 격차를 5.0→3.3으로 줄인 튜닝의 결과다 — 2부에서 상술)
- **창의성은 사건과 관계에 가점** (4단계): 단순히 마법 단어·원소·수식어를 많이 넣었다고 위력이 오르지 않는다. 충돌·융합·개화·붕괴·메아리처럼 화면에 옮길 수 있는 관계와 변화가 실제로 연결될 때 form·병렬·sequence와 제한된 power 가점으로 보상한다.
- **말을 통제 가능한 전투 안무로** (5~8단계): "분신을 지그재그로 돌진"은 소환수 내부 behavior로, "화염벽을 삼각형으로"는 wall shape로, "두 번 울리고 잠시 뒤 다시"는 form/wait 시간축으로 펼쳐진다. 플레이어 위치를 LLM이 강제로 바꾸는 move 는 제거했고, 이동 이미지는 빠른 slash·beam·잔상·도착 효과 같은 form으로 번역한다. 일반 영창과 필살영창을 분리해 자유로운 표현과 전투 주도권을 함께 지킨다.

1.4 "LLM을 신뢰하지 않는" 설계

LLM의 출력은 **신뢰할 수 없는 입력**으로 취급한다. 게임의 밸런스·안전·재현성이 모델의 번덕에 좌우되면 안 되기 때문이다. 세 겹의 방어가 있다.

① **스키마 재검증·클램프** (src/spell/validate.ts, src/spell/spellPlanValidate.ts) 프록시가 돌려준 JSON을 게임이 그대로 쓰지 않는다. validateJudgement와 validateSpellPlan이 다시 검증해 — effect/element/form이 허용 목록(enum) 안에 있는지, 일반 영창이 최대 2원소인지, plan behavior가 form/wait 인지, power-duration-sequence 수가 일반/필살 계약 범위인지 — 확인한다. 소환 behavior와 wall shape도 전용 검증기가 수치·종류를 제한한다. move나 power: 9999처럼 현재 계약 밖의 출력은 게임에 반영되지 않는다.

② **서버측 프롬프트 고정** (proxy/worker.js) 프롬프트와 API 키는 워커에 있고, 클라이언트는 { text, castMode, resonance? }만 보낸다. 플레이어가 브라우저에서 프롬프트를 바꿔 "power 100 고정" 같은 조작을 하는 것을 원천 차단한다.

③ **무중단 폴백** (src/spell/mockJudge.ts) 프록시 장애·타임아웃·무료 티어 한도 소진(429) 등 어떤 이유로든 원격 판정이 실패하면, 로컬 규칙 기반 판정기 MockJudge가 즉시 대신한다. 판정 품질은 다소 낮아질 수 있어도 **게임은 절대 멈추지 않는다**. 심사위원이 몰려 무료 한도를 소진해도 게임은 계속 플레이 가능하다.

요약하면 LLM은 **의미 해석**만 담당하고, 그 출력의 안전·범위·재현성은 전부 결정론적 코드가 통제한다. "창의성은 LLM에, 밸런스는 코드에."

1.5 행동 설계의 경계: 소환 behavior·벽 shape·form/wait 시간축

가장 앞선 활용은 LLM이 소환수의 움직임을 직접 설계하는 것이다. "분신을 만들어 지그재그로 돌진시켜라" 같은 입력에서, LLM은 소환 주문임을 인식하고 **움직임 프로그램(behavior)**을 함께 생성한다.

움직임은 6개의 부품(kind)으로 이루어진 작은 DSL로 표현된다:

kind	의미
orbit	플레이어 주위를 선회
chase	표적을 추적
dash	표적으로 돌진
zigzag	갈지자로 접근
hold	제자리 대기
retreat	표적 반대로 후퇴

"A 하다가 B" 같은 순차 묘사는 steps 배열의 순서로 표현된다. 예를 들어 "지그재그로 접근하다 돌진" → [zigzag, dash]:

```
"behavior": {
  "steps": [
    { "kind": "zigzag", "seconds": 2, "speed": 300, "amplitude": 60 },
    { "kind": "dash", "seconds": 1, "speed": 440 }
  ],
  "loop": false
}
```

여기서도 코드가 상한을 강제한다: seconds 1~6, speed 1~460, orbit.radius 1~150, zigzag.amplitude 1~100, steps 최대 6개. LLM이 과한 수치를 뱉어도 밸런스가 깨지지 않는다. 이 DSL 덕분에 플레이어는 "정해진 소환수"가 아니라 **자기가 말로 설계한 움직임**을 보게 된다 — "LLM이 게임 행동을 설계한다"가 한 것에 보이는 지점이다.

형상도 같은 방식으로 — 벽의 모양 설계 (프롬프트 6항)

움직임(behavior)이 "어떻게 움직이는가"를 열었다면, 같은 원리로 벽(form: wall)의 **모양**도 플레이어의 말로 설계한다. "화염의 벽을 지그재그로 세워라"에서 "지그재그"는 예전엔 조용히 버려졌다(벽이 고정 원호 하나뿐이었다). 이제 LLM이 shape를 함께 설계한다:

- 형상 6종: arc (원호)·line (직선)·zigzag (갈지자)·wave (물결)·ring (단힌 원)·polygon (다각형).
- "원을 그리며 둘러싸라" → ring, "삼각형으로" → polygon. 특정 단어를 하드코딩하지 않고 **모양 자체를 어휘로** 연다.

- 같은 안전벽: 화이트리스트 밖은 거부→기본 원호 폴백, 수치 클램프. **형상은 위력이 아니다** — 어떤 모양이든 벽이 막는 정면폭은 크기(size)가 정한 값으로 정규화된다(모양은 표현일 뿐).

이 기능은 "만들기 전에 측정한다" 원칙으로 검증했다: 실 Gemini 배포 후 모양 묘사 문장의 **형상 반영 10/10**, 모양 묘사가 없을 때 오답 **0/6**(원호 유지). "LLM이 게임의 형태를 설계한다"가 소환 행동에 이어 벽에서도 성립한다.

플레이어 이동 DSL은 제거하고, 사건의 시간축만 남겼다

초기 plan은 `form·move·wait` 를 모두 허용했다. 그러나 LLM이 만든 `move` 가 플레이어의 WASD 조작권과 충돌하고, 전투 위치를 예측하기 어렵게 만드는 문제가 있어 #319에서 제거했다. 지금 plan은 다음 두 부품만 쓴다.

- `form`: 피해·회복·보호·소환 등 실제 게임 효과. 같은 sequence 안의 form은 동시에 실행된다.
- `wait`: 다음 사건까지의 박자. `form → wait → form` 으로 반복·지연·변화를 표현한다.

따라서 "파고들어 벤다"의 이동감은 플레이어 좌표를 강제로 옮기는 대신 빠른 slash, beam, 잔상·도착 효과 같은 **form의 시각 언어**로 번역한다. 반면 소환수의 `behavior` 는 소환된 개체 내부 움직임이므로 유지된다. 둘을 분리해 **플레이어 조작권은 코드가 지키고, 주문의 형태와 리듬은 AI가 설계하게** 했다.

1.6 기억하는 보스 — 도발 대사 + 미리 캐스트

최종 보스는 플레이어의 **지난 전적을 기억**하고, LLM이 그에 맞는 도발 대사를 생성한다. 프록시의 `/boss-line` 엔드포인트가 런 요약(JSON)을 받아 1~2문장의 대사를 돌려준다.

```
플레이어 전적: { favoriteElement: "fire", topSpellName: "화염 대폭발",
                deaths: 3, clears: 0 }
                |
                ▼ (/boss-line, LLM)
보스 대사: "또 불이냐. 화염 대폭발도 세 번은 통하지 않아."
```

프롬프트는 애용 원소·최고 주문명·사망 횟수를 자연스럽게 비교도록, 첫 조우(사망·클리어 모두 0)면 낮선 도전자를 알보는 톤으로 지시한다. 실패 시 클라이언트가 템플릿 대사로 폴백하므로 여기서도 무중단이 보장된다.

미리 캐스트 — 보스가 당신의 주문을 되꾼다 (AI 루프가 달하는 곳)

도발이 말로 기억한다면, 미리 캐스트는 **행동으로 기억한다**. 최종 보스는 이번 런에서 플레이어가 만든 **가장 강한 주문을 꺼내 그 모습 그대로 되돌려 시전**한다. 렌더러(`castSpell`)가 시전자 중립이라 — 플레이어가 자유 영창으로 빚은 그 이펙트가, 한 픽셀도 안 고친 채 자신에게 날아온다.

여기서 **AI 루프가 달한다**: 정해진 스킬이 아니라 **LLM이 당신의 문장을 해석해 만든 주문**이 보스의 무기가 된다. "보스가 나를 기억한다"가 내성 수치·텍스트 한 줄이 아니라 **내가 발명한 마법이 나에게 돌아오는 장면**으로 읽힌다. (밸런스 가드: 위력 0.35배 + 재사용 쿨다운, 즉발 품은 주문명을 호명하는 텔레그래프 — "보스가 『x』를 역영창한다" — 로 회피 유예를 준다. 이동 시간이 있는 품은 WASD로 피한다.)

보스의 원소 마법·시야 장막·중력 인력(비전 마법)은 미리 캐스트가 깬 인프라(시전자 중립 렌더)를 재사용하지만, **고정 수치 스크립트**이지 LLM 생성이 아니다 — 이 문서(게임 속 AI)의 범위 밖이다.

1.7 진화·융합 작명

주문이 진화하거나 정령이 융합할 때, 격상된 이름을 LLM이 짓는다(`/evolve-name`). 예: fire+lightning 융합 → "작열하는 뇌운". 12자 이내의 함축적 이름 하나만 생성하며, 이 결과도 localStorage에 캐시된다(같은 조합 = 같은 이름, 재현성·호출 절감). 실패 시 템플릿 이름으로 폴백한다.

1.8 인프라 — Cloudflare Worker 프록시

모든 LLM 호출은 Cloudflare Worker 프록시(`proxy/worker.js`)를 경유한다. 프록시의 역할은 네 가지:

1. **API 키 은닉** — Gemini API 키는 워커 시크릿(`GEMINI_API_KEY`)에 있고 클라이언트에 절대 노출되지 않는다.
2. **레이트리밋** — IP·인스턴스별 분당 15회(`RATE_LIMIT_PER_MIN = 15`)로 남용과 순간 호출 폭주를 막는다.
3. **CORS** — 배포 오리진(GitHub Pages)과 로컬 개발(vite dev, 유동 포트) 요청만 허용한다.

4. 프롬프트 서버측 고정 — 1.3~1.7의 모든 프롬프트가 워커에 있다.

배포 설정은 Worker 실행 위치를 `gcp:asia-northeast1` 로 지정한다(`proxy/wrangler.toml`). Gemini 지역 제한 오류가 배포 경로에 따라 간헐적으로 발생했던 뒤, 실제 데모 Worker와 로컬 실험 설정의 배치 차이를 없애기 위해 반영한 운영 안정화 조치다.

모델 핀 정책 (중요): 모델은 `gemini-3.5-flash-lite` 로 명시 고정한다. 초기에 `gemini-flash-lite-latest` 별칭을 썼다가, 이 별칭이 심사 기간 중 `2.5 → 3.1 → 3.5` 로 자동 드리프트하며 요청 규격(예: Gemini 3.5부터 `temperature / thinkingBudget` 폐기)이 바뀌는 문제를 겪었다. 재현성을 위해 `-latest` 별칭을 금지하고 버전을 못박았다. (이 사건의 디버깅 교훈은 2부에서 상술)

판정 요청의 생성 설정은 `maxOutputTokens: 2048`, `responseMimeType: 'application/json'` 이다. 모델이 코드펜스("```json ") 등 군더더기를 붙일 수 있으므로, 워커는 응답에서 첫 {~마지막 } 구간만 추출해 파싱을 확인한 뒤 반환한다(검증은 클라이언트에서 한 번 더).

2부. AI로 만든 게임

INCANT는 "AI를 게임에 넣은" 것에 그치지 않고, **게임 자체를 AI 에이전트로 만들었다**. 3인 팀이 각자 코딩 에이전트를 지휘해 판정 시스템·전투·연출·아트·사운드를 구현했고, 모든 유의미한 작업을 `docs/AI_USAGE_LOG.md` 에 즉시 1줄씩 기록했다 — 2026-08-09 기준 **386건**이 누적돼 있다.

2.1 사용한 AI 도구

도구	용도	주 담당 영역
Claude Code (Opus 4.8)	코드 구현·디버깅·아트 파이프라인	AI 판정 시스템, 시각 연출, 시스템 코드
OpenAI Codex	코드 구현	런 구조, 전투 폼, 보스 패턴, 통합 폴리싱
Google Gemini (Nano Banana 2)	이미지 생성	적·플레이어 스프라이트, 보스방 배경
Adobe Firefly	오디오 생성	효과음(Generate Sound Effects), BGM(Generate Soundtrack)

로그 386건의 도구 분포는 **Codex 계열 348건, Claude Code 계열 38건**으로, 두 코딩 에이전트를 병행했다(후반부는 Codex 비중이 커졌다). 이미지는 Gemini, 사운드는 Firefly가 전담했다(각 상세는 3부).

2.2 협업 워크플로

- **에이전트 병렬 지휘:** 3인이 각자 다른 에이전트(Claude Code / Codex)를 동시에 몰며, GitHub 이슈·PR로 작업을 분리했다.
- **계약 우선(contract-first):** 시스템 간 인터페이스(예: 보상 종류 `RewardKind`, 판정 스키마)를 먼저 커밋해 계약을 고정하고, 그 위에서 각자 병렬로 구현했다 — 에이전트 간 충돌을 줄이는 핵심.
- **회귀로 방어:** 각 기능마다 순수 로직을 회귀 스크립트로 고정(`test:judge`, `test:grimoire`, `test:spirit` 등)해, 다른 에이전트의 변경이 기존 계약을 깨면 즉시 드러나게 했다.
- **즉시 로깅:** "나중에 몰아 쓰면 디테일이 사라진다"는 규칙 아래, 작업 직후 프롬프트 요약·산출물·교훈을 로그에 남겼다.

2.3 대표 사례 — 프롬프트에서 산출물까지

로그에서 네 가지 사례를 골랐다. 공통점은 ****"AI가 낸 결과를 곧이곧대로 믿지 않고, 실제 값으로 검증했다"**는 것이다.

① **판정기 안정화 — 모델 드리프트와 인코딩 오진** 판정 프록시가 "화염구" 에도 `fizzle` 을 내는 위기를 겪었다. 원인 추적 중 두 가지가 겹쳐 있었다: (a) `gemini-flash-lite-latest` 별칭이 `2.5→3.1→3.5` 로 자동 드리프트하며 폐기된 모델이 404를 냈고, (b) **Windows Git Bash**에서 `curl` 로 한글을 인라인 전송하면 **UTF-8이 깨져** 서버가 깨진 문자열을 난센스로 판정하고 있었다. 후자를 "모델이 짧은 한글을 못 읽는다"로 오진해 여러 날 모델 교체를 헛돌았다. **교훈:** 게임(브라우저)은 항상 정상

UTF-8을 보내 실제로는 멀쩡했다. Node로 파일 바디를 만들어 재현하니 프록시가 9/9 정확히 판정 — **테스트 도구 자체가 오염원**이었다. 이후 모델을 `gemini-3.5-flash-lite`로 명시 편했다.

② **스프라이트 누끼** — "**Gemini는 투명 PNG를 못 준다**" 몸·플레이어 스프라이트 7종을 Gemini로 생성했는데, **모든 출력이 알파 0%**(투명 대신 체커보드를 픽셀로 그려 옴)였다. Claude Code가 배경 톤을 자동 검출하고 테두리부터 flood-fill로 누끼를 따 해결했다. 또 총괄의 지적 **"틴트 때문에 몸이 그 색으로만 보인다"**가 정확해서 — `setTint`는 곱셈이라 재질감이 죽는다 — **재질 텍스처와 발광 마스크를 2장으로 분리해**, 색은 발광이 전담하고 재질은 보존하도록 렌더 파이프라인을 바꿨다 (`src/render/spriteLayers.ts`).

③ **프롬프트 튜닝의 편향 정정** — **held-out 검증** 판정 프롬프트를 튜닝하던 중, 사용자가 **"네 프롬프트 예시에만 반응하게 편향된 것 아니냐"**고 지적했다. 재점검하니 프롬프트에 코퍼스 문구(숲의 분노)를 예시로 박아두고 **같은 문구로 검증**하는 teaching-to-the-test였다. 예시를 제거하고 순수 원리로 교체한 뒤, **프롬프트·코퍼스에 없는 held-out 3쌍**으로 재검증하니 교차언어 위력 격차가 5.0(오염)→3.3(무오염)으로 드러났다. **교훈**: LLM 품질 주장은 튜닝 예시와 분리된 데이터로, 다회 평균으로 검증해야 한다.

④ **배경이 안 뜨던 근본 원인** — **로더 경로 오염** 배경이 안 뜨는 버그를 포맷 문제로 오판해 webp→jpg→png로 세 번 바뀌도 실패했다. network 탭에서 **Phaser가 실제로 요청한 URL**을 보고서야 원인을 잡았다: 오디오 로더가 설정한 `setPath`가 되돌려지지 않아 배경 요청 경로가 오염됐고, 그 경로에 Vite가 `index.html`을 200으로 반환해 Phaser가 HTML을 이미지로 처리하려다 실패한 것이었다. **교훈**: `fetch`가 200이라고 믿지 말고 로더가 실제로 요청한 URL을 볼 것.

2.4 배운 점 — 에이전트 지휘 노하우

- **"초록불"을 믿지 말고 실제 값으로 검증한다.** 통과하는 테스트가 아니라 held-out 데이터·다회 평균·network 탭 실측·실제 조회로 의도가 맞는지 본다. 위 4개 사례가 전부 "그럴듯한 결과"를 한 겹 더 파고들어 잡은 것이다.
- **도구 경계에 함정이 있다.** Windows `curl`의 UTF-8, Gemini의 알파 부재, `fetch` 200의 착시 — 버그는 종종 우리 코드가 아니라 도구와 코드의 틈에 숨는다.
- **창의성은 LLM에, 밸런스·안전·재현성은 코드에.** 게임 안에서든(1부) 개발 과정에서든 같은 원칙을 지켰다 — AI의 판단은 반드시 검증·클램프를 거친다.
- **계약을 먼저 고정하면 에이전트를 병렬로 몰 수 있다.** 인터페이스를 커밋으로 못박고 회귀로 지키는 것이, 여러 에이전트의 동시 작업을 충돌 없이 합치는 열쇠였다.

3부. 외부 에셋·오픈소스 출처

전체 에셋별 생성 프롬프트·선정 근거·라이선스 원문·후처리 상세는 저장소의 `docs/ASSET_CREDITS.md`에 기록돼 있다. 아래는 출처·라이선스 요약이다.

3.1 오디오 — Adobe Firefly

- **효과음**: Adobe Firefly — *Generate Sound Effects*
- **BGM**: Adobe Firefly — *Generate Soundtrack (Beta)*
- **라이선스**: Adobe 공식 안내상 Generate Soundtrack 결과물은 royalty-free·상업적 사용 가능하며, Firefly FAQ는 Beta 기능도 별도 금지가 없는 한 상업 프로젝트에 사용 가능하다고 안내한다. (이용 조건 확인일 2026-07-17)
 - 효과음: <https://www.adobe.com/products/firefly/features/sound-effect-generator.html>
 - 사운드트랙: <https://www.adobe.com/products/firefly/features/ai-music-generator.html>
 - Firefly Beta 상업 이용: <https://helpx.adobe.com/firefly/web/get-started/learn-the-basics/adobe-firefly-faq.html>
- **후처리**: 저장소의 `scripts/process-audio-assets.py` · `scripts/create-bgm-loop.py`로 무음 정리·페이드·피크 정규화·루프 제작. 게임 경로 `public/assets/audio/`.

3.2 이미지·스프라이트 — Google Gemini (Nano Banana 2)

- **용도:** 적 6중·플레이어 스프라이트, 보스방/스테이지 배경. 게임 경로 `public/assets/sprites/`, `public/assets/backgrounds/`.
- **생성:** 게임 실제 엔티티와 탑다운 시점·색 구분 체계에 맞춘 프롬프트를 Claude Code가 설계하고 Gemini로 생성.
- **후처리(Claude Code):** ① 출력 우하단의 Gemini 워터마크를 거울 패치 페더링으로 제거 ② 투명 대신 그려진 체커보드 배경을 누끼 처리 ③ 재질/발광 텍스처 2장 분리. AI 생성 원본은 별도 보관하고 저장소에는 채택본만 포함.

3.3 폰트 — Noto Serif KR

- **용도:** 타이틀 카피·주문명 각인.
- **라이선스:** SIL Open Font License 1.1. 원문을 `public/assets/fonts/OFL-NotoSerifKR.txt` 에 포함.
- **출처:** <https://github.com/google/fonts/tree/main/ofl/notoserifkr> (Google Fonts 공식 가변 TTF에서 필요한 자모만 서브셋한 WOFF2를 자체 호스팅).

3.4 프로젝트 자체 제작

- `public/assets/og-incant.svg`: 타이틀 화면의 색상·마법진·타이포그래피를 재사용한 1200×630 OG 이미지 — 팀 자체 제작.

3.5 오픈소스 라이브러리

라이브러리	버전	라이선스	용도
Phaser	3.90	MIT	게임 엔진(렌더·입력·썸)
TypeScript	5.9	Apache-2.0	언어·타입 검사
Vite	7.1	MIT	빌드·개발 서버

인프라: Cloudflare Workers(판정 프록시 호스팅), GitHub Pages(웹 빌드 배포).